

INFORME FINAL

Integración de K-Means con un Algoritmo Genético para JSSP

Inicialización informada de la población, comparación experimental A-D y análisis de resultados

Integrantes	Stack tecnologico
Chiu Flores Fernando	Codex (chat gpt 5.5)
Diaz Ibarra Cristian Jassiel	Antigravity (gemini 3.5 flash, claude),claude
Hernandez Olvera Jesus Javier	Antigravity (gemini 3.5 flash, claude)
Santiago Felix Maximiliano	Antigravity (gemini 3.5 flash, claude), Open Code (orquestacion multiagente)

Lenguaje de ejecución: Python (scikit-learn y NumPy)

Ejecuciones analizadas: 36 corridas: 3 casos x 4 configuraciones x 3 semillas



Licencia académica. Este trabajo se presenta con fines académicos. Se permite su consulta y reproducción dentro del curso, citando la autoría correspondiente y sin uso comercial.

Nota de entrega. Antes de presentar el archivo, se deben completar los datos de portada y anexar las capturas requeridas por el profesor con fecha visible.



Resumen ejecutivo

El presente proyecto integra K-Means con un algoritmo genético (AG) para el problema de programación Job Shop Scheduling Problem (JSSP). La integración se restringe a la creación de la población inicial: K-Means agrupa trabajos según perfiles de carga o uso de máquinas, y reglas de despacho convierten esos grupos en órdenes compatibles con el cromosoma existente. La evaluación por makespan y los operadores principales del AG se conservan.

Se ejecutaron cuatro configuraciones de inicialización (A-D) sobre tres tamaños de instancia (Base, Mediano y Grande), usando tres semillas por combinación, para un total de 36 corridas. La implementación se realizó en Python con KMeans de scikit-learn, inicialización k-means++ y semillas reproducibles.

Resultado principal. La configuración informada no produjo una única variante ganadora en todos los tamaños: D fue la mejor en promedio para Base, C para Mediano y B para Grande. En las 36 corridas no se observaron soluciones infactibles, por lo que la integración preservó la representación válida del cromosoma.

Contenido del informe

- Problema, objetivo y criterio de evaluación.
- Fundamento de K-Means y estrategia de integración con el AG.
- Diseño experimental: parámetros, configuraciones y casos.
- Implementación y reutilización funcional del código previo.
- Resultados, convergencia, discusión y conclusiones.
- Reproducibilidad, bitácora de IA y pendientes de entrega.

1. Problema y objetivo

El JSSP busca programar operaciones de distintos trabajos sobre máquinas compartidas, respetando el orden tecnológico de cada trabajo y evitando que una máquina procese más de una operación simultáneamente. El criterio usado es minimizar el makespan, es decir, el tiempo de terminación de la última operación programada.

El objetivo experimental es determinar si una población inicial parcialmente informada mediante K-Means y reglas de prioridad puede mejorar el desempeño del AG respecto de una población completamente aleatoria, sin modificar la representación del cromosoma ni los operadores genéticos centrales.

Restricción de diseño. K-Means no resuelve el calendario del JSSP. Su función en este proyecto es producir agrupamientos y órdenes iniciales de trabajos; el AG continúa realizando la búsqueda de soluciones y la evaluación final mediante makespan.



2. Fundamento de K-Means

De acuerdo con el material de clase, K-Means es un método de agrupamiento no supervisado que parte de K centroides, asigna cada objeto al centroe más cercano, recalcula los centroides y repite el proceso hasta alcanzar estabilidad. En el proyecto, los objetos son los trabajos del JSSP y sus características son vectores numéricos derivados de las máquinas y tiempos requeridos.

Elemento	Aplicación en el proyecto
K	Se fijó $K = 3$ para representar grupos de trabajos con comportamiento diferenciado.
Inicialización	KMeans de scikit-learn con <code>init = k-means++</code> , <code>n_init = 10</code> y semilla fija.
Perfil 1	Carga por máquina: vector de tiempos de procesamiento de cada trabajo.
Perfil 2	Uso de máquinas: vector binario que registra si cada máquina es utilizada.
Salida usada	Etiquetas de cluster; después se ordenan trabajos mediante reglas de prioridad.

La elección de conservar una proporción aleatoria en las configuraciones informadas responde a una limitación conocida de K-Means: la partición puede favorecer estructuras locales y reducir diversidad. La mezcla busca aprovechar información inicial sin cerrar prematuramente la exploración del AG.

3. Integración K-Means + Algoritmo Genético

3.1 Representación y operadores conservados

El cromosoma se mantiene como una matriz máquinas por trabajos; cada fila contiene una permutación válida de los trabajos. La evaluación de makespan, selección por torneo, cruza PMX, mutación dinámica y elitismo proceden de la lógica base del AG-JSSP. La modificación funcional se concentra en generar una parte de la población inicial a partir de órdenes informados.

Componente	Tratamiento en la solución
Representación del cromosoma	Se conserva; cada fila continúa siendo una permutación válida.
Makespan	Se reutiliza la función de evaluación del código Python previo.
Cruza PMX	Se reutiliza del código Python previo.
K-Means	Se integra con <code>sklearn.cluster.KMeans</code> ; no se implementa desde cero.
Instancias adicionales	Se generan en un módulo funcional separado con semillas registradas.



3.2 Reglas y diversidad controlada

Regla	Criterio empleado	Observación
SPT	Carga total ascendente.	Prioriza trabajos cortos.
LPT	Carga total descendente.	Prioriza trabajos largos.
MWR	Trabajo restante descendente.	Al inicio se aproxima a LPT, pero se conserva como regla conceptualmente distinta.
MOPNR	Número de operaciones restantes descendente.	Desempata por carga total descendente.

Para evitar individuos idénticos, los órdenes base se perturban con una lista restringida de candidatos, intercambio, inversión corta o rotación. Estas variantes cambian la prioridad inicial, pero no alteran la condición de permutación exigida por el cromosoma.

4. Diseño experimental

4.1 Parámetros ejecutados

Parámetro	Valor ejecutado	Criterio
K de K-Means	3	Partición compacta para los perfiles de trabajo.
Inicialización	k-means++, n_init = 10	Uso de librería estándar y estabilidad de inicialización.
Semillas del AG	1, 2 y 3	Tres corridas reproducibles por combinación.
Población	20 individuos	Valor conservado del algoritmo base.
Generaciones	50	Valor conservado del algoritmo base.
Selección	Torneo, k = 3	Operador conservado del AG.
Lenguaje ejecutado	Python	Implementación consolidada para los resultados presentados.

4.2 Configuraciones comparadas

Config.	Composición de población inicial	Función experimental
A	100% aleatoria	Línea base.
B	70% Perfil 1 + SPT; 30% aleatoria	Inicialización informada simple.
C	40% Perfil 1 + SPT; 40% Perfil 2 + SPT; 20% aleatoria	Compara perfiles

Config.	Composición de población inicial	Función experimental
		manteniendo SPT.
D	20% Perfil 1 + SPT; 20% Perfil 1 + LPT; 20% Perfil 2 + MWR; 20% Perfil 2 + MOPNR; 20% aleatoria	Combina reglas y perfiles.

4.3 Instancias

Caso	Trabajos x máquinas	Operaciones por trabajo	Semilla de instancia
Base	8 x 14	1 a 5	proporcionada
Mediano	15 x 14	5 a 7	1514
Grande	30 x 20	8 a 10	3020

El caso Base conserva los datos proporcionados en el código previo. Los casos Mediano y Grande se generaron de manera reproducible y se almacenaron en archivos CSV de secuencias y tiempos, junto con un manifiesto de parámetros.

5. Resultados experimentales

Se ejecutaron 36 corridas (tres casos, cuatro configuraciones y tres semillas). La tabla presenta el mejor makespan observado, el promedio de makespan, su desviación estándar y el tiempo promedio por configuración. En todos los casos un makespan menor representa mejor desempeño.

Caso	Config.	Mejor	Promedio	Desv. est.	Tiempo (s)	Infactibles
Base	A	280	293.33	11.55	0.724	0.00%
Base	B	300	300.00	0.00	0.762	0.00%
Base	C	280	293.33	11.55	0.734	0.00%
Base	D	280	286.67	11.55	0.675	0.00%
Mediano	A	800	830.33	30.50	1.511	0.00%
Mediano	B	811	841.67	26.56	1.453	0.00%
Mediano	C	778	804.67	25.17	1.486	0.00%
Mediano	D	791	807.33	18.88	1.684	0.00%
Grande	A	1767	1834.00	59.57	5.489	0.00%
Grande	B	1674	1737.33	69.08	5.017	0.00%



Caso	Config.	Mejor	Promedio	Desv. est.	Tiempo (s)	Infactibles
Grande	C	1714	1743.67	46.29	5.160	0.00%
Grande	D	1745	1764.67	19.50	4.923	0.00%

Tabla 1. Resultados agregados de 36 corridas. Filas verdes: mejor promedio por caso. Fuente: elaboración propia.

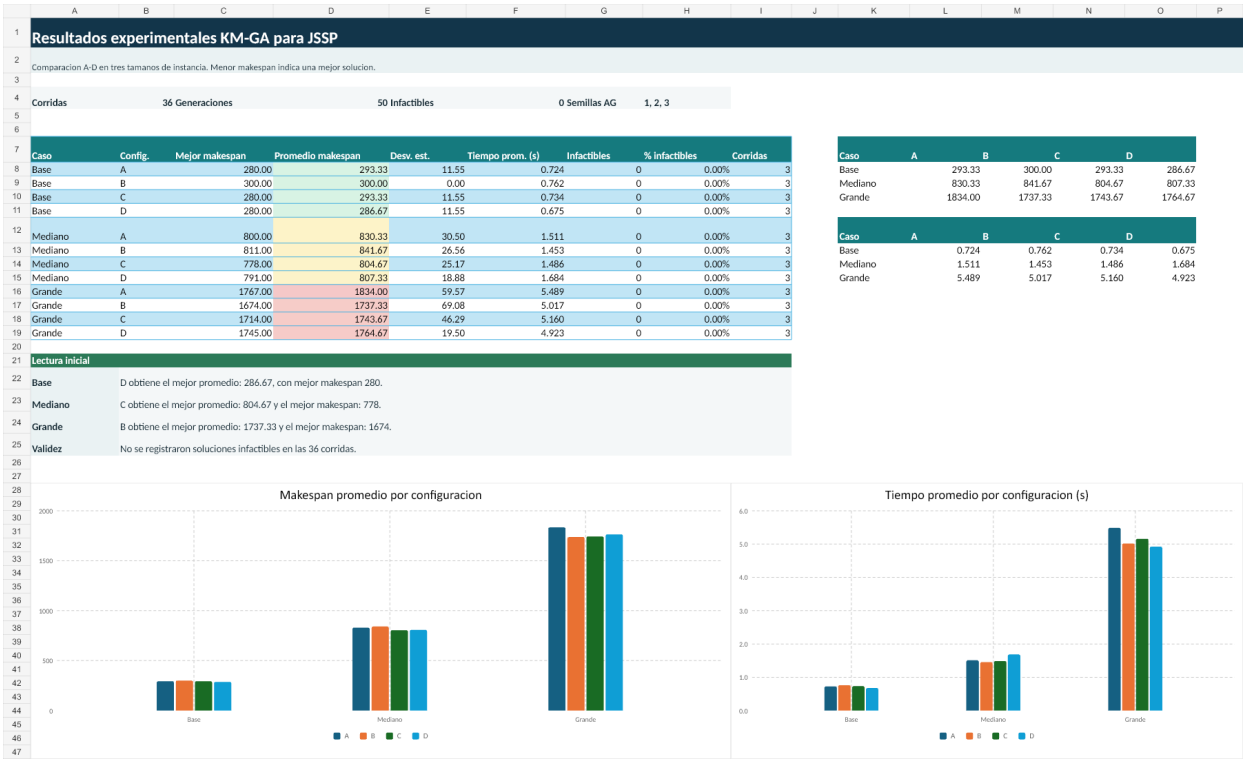


Figura 1. Resumen comparativo de makespan y tiempo promedio por configuración. Fuente: elaboración propia.

5.1 Lectura por tamaño de instancia

- Caso Base: D logró el menor promedio (286.67); A, C y D alcanzaron un mejor makespan de 280.
- Caso Mediano: C logró el menor promedio (804.67) y el menor mejor makespan (778).
- Caso Grande: B logró el menor promedio (1737.33) y el menor mejor makespan (1674).
- Validez: no se registraron soluciones infactibles en ninguna de las 36 corridas.

6. Convergencia de corridas representativas

Para observar el comportamiento durante las 50 generaciones, se compara la configuración A contra la configuración de menor promedio de cada caso usando la misma semilla del AG (semilla 3). Estas figuras son ilustrativas de convergencia y complementan, sin sustituir, los promedios de las tres corridas.

6.1 Caso Base

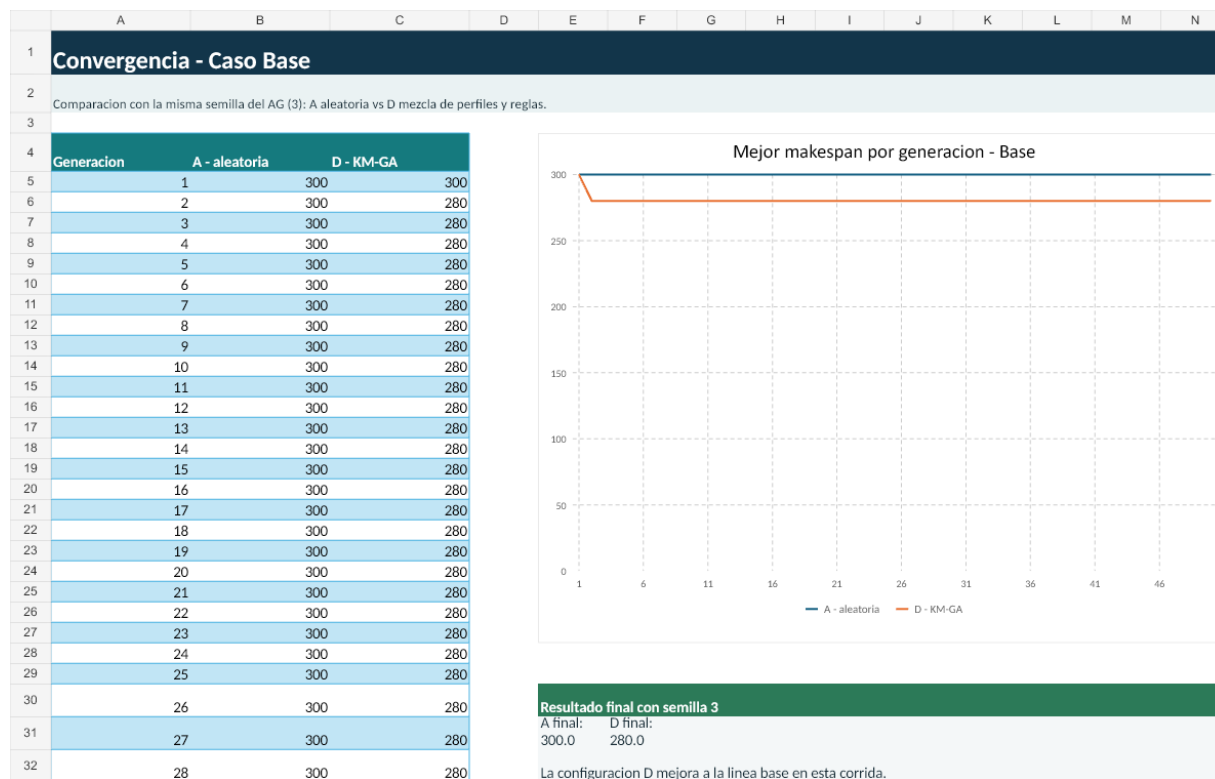
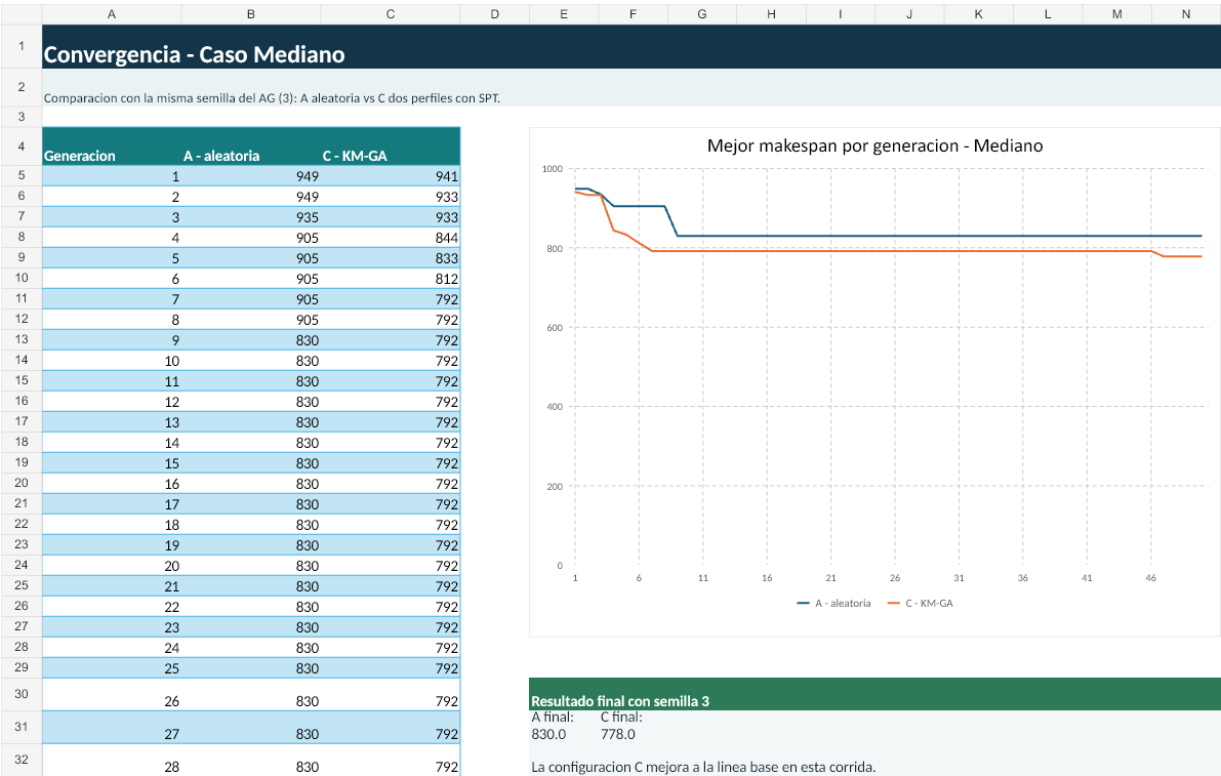


Figura 2. Convergencia representativa para el caso Base, semilla 3. Fuente: elaboración propia.

En Base, D mejora de 300 a 280 con rapidez, mientras A permanece en 300 en la corrida mostrada.



6.2 Caso Mediano



6.3 Caso Grande

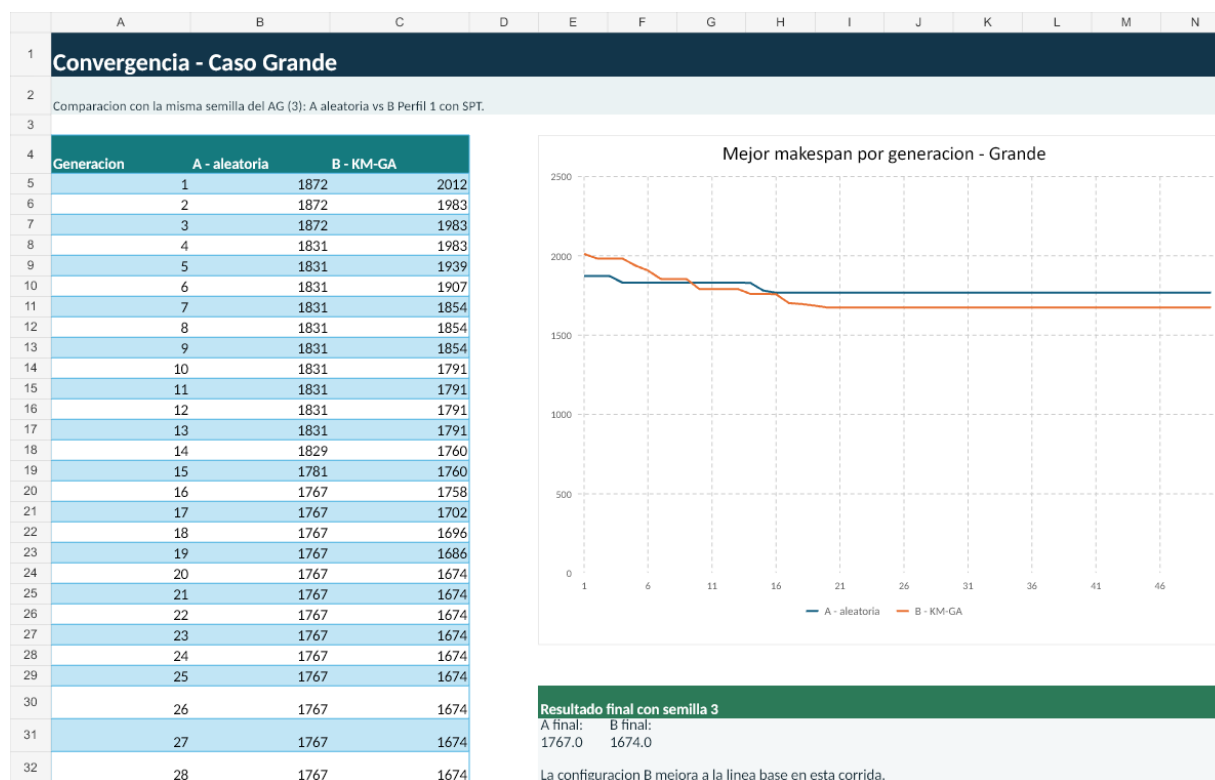


Figura 4. Convergencia representativa para el caso Grande, semilla 3. Fuente: elaboración propia.

En Grande, B supera gradualmente a A y concluye en 1674 frente a 1767.

7. Discusión

Los resultados apoyan la utilidad de una inicialización informada, pero no indican que una sola configuración sea universalmente superior. En el caso Base, la combinación diversa de D fue favorable; en el caso Mediano, el uso conjunto de los dos perfiles con SPT en C mostró el mejor comportamiento; en el caso Grande, una solución más simple, B, produjo el menor promedio y el mejor makespan.

Este comportamiento es consistente con la naturaleza del JSSP: el beneficio de agrupar por carga o uso de máquinas depende de la estructura y tamaño de la instancia. K-Means aporta una guía inicial, pero el AG aún necesita diversidad para explorar órdenes alternativos.

La ausencia de soluciones infactibles es un resultado metodológicamente importante: la introducción de K-Means no rompió el formato de los cromosomas. Esto confirma la decisión de limitar la integración a la población inicial y reutilizar los operadores ya existentes.

Limitación. Los resultados reportados se obtuvieron con tres semillas por configuración y con la implementación Python. Para una comparación estadística más robusta podría



ampliarse a cinco o más semillas y, si se solicita explícitamente, ejecutar una validación equivalente en R.

8. Reproducibilidad y entregables técnicos

La solución se organizó de manera funcional: un módulo produce las instancias y otro ejecuta el experimento KM-GA. Se reutilizaron las funciones de makespan y cruza PMX del código Python previo proporcionado para el curso, incorporando funciones nuevas solamente para los perfiles, K-Means, configuraciones de población y exportación de resultados.

Archivo o carpeta	Contenido verificable
generador_instancias_jssp.py	Crea casos Base, Mediano y Grande y exporta CSV reproducibles.
km_ga_jssp_experimento.py	Ejecuta A-D, tres semillas, KMeans y guarda resultados/convergencia.
instancias_jssp/	Matrices, secuencias y manifiesto de semillas para los tres casos.
resultados_km_ga_jssp_tres_casos/	CSV de 36 corridas, convergencia y resumen estadístico.
Resultados_experimentales_KM_GA_JSSP.xlsx	Libro con tablas, resumen y gráficas finales.

8.1 Evidencias que deben acompañar la entrega

- Captura de la ejecución del experimento en Python con fecha o nombre fechado visible.
- Captura del archivo Excel abierto en la hoja Resumen y de las gráficas de convergencia.
- Captura o listado que evidencie el uso de `sklearn.cluster.KMeans`.
- Cuando el profesor exija ejecución en ambos lenguajes, adjuntar evidencia adicional de R; los resultados de este informe corresponden a Python.

9. Bitácora de uso de inteligencia artificial

Se utilizó una herramienta de asistencia con IA como apoyo para organizar requerimientos, revisar materiales proporcionados, estructurar el protocolo experimental, integrar funciones del código previo, generar archivos de salida y preparar la documentación. Las decisiones experimentales y los resultados presentados se sostienen en ejecuciones reproducibles y archivos verificables.

Actividad asistida	Resultado / validación
Análisis de instrucciones y notas del profesor	Se identificaron parámetros, evidencia y entregables requeridos.
Revisión de materiales K-Means,	Se priorizó la reutilización de funciones previas compatibles.



Actividad asistida	Resultado / validación
MWR y AG-JSSP	
Integración Python KM-GA	Se usó KMeans de scikit-learn y se conservaron makespan y PMX.
Diseño experimental A-D	Se ejecutaron 36 corridas en tres tamaños de instancia.
Documentación de resultados	Se generaron CSV, Excel, gráficas y el presente informe.

10. Conclusiones

La integración de K-Means en la creación de la población inicial del algoritmo genético es funcional para el JSSP evaluado. La estrategia conserva la validez del cromosoma y permite comparar distintas combinaciones de perfiles, reglas de ordenamiento y diversidad aleatoria.

Los experimentos muestran que las configuraciones informadas pueden superar a la línea base, especialmente en los casos Mediano y Grande, aunque la configuración favorable depende de la instancia. Por ello, el resultado central no es seleccionar una mezcla universal, sino mostrar que la inicialización puede mejorarse de manera controlada, modular y reproducible.

Como continuación natural se propone ampliar el número de semillas, documentar las capturas finales solicitadas y, solo si la evaluación lo requiere, replicar las corridas en R para contrastar lenguajes.

Fuentes y materiales utilizados

- Material de clase: 4-1 Kmeans algoritmo.pdf. Presentación convertida a PDF proporcionada para el curso.
- Material de clase: Most Work Remaining en JSSP.pptx. Ejemplos y regla MWR proporcionados para el curso.
- Código de referencia: T8 AGS_JSSP Lineas equivalentes a T6-R.py y T9 AGS_JSSP Lineas equivalentes a T7-R.py.
- Código de referencia: T7 AGS_JSSP Lineas 133-277.R y Rosenbrock_ga.R.
- Enunciado del proyecto: proyecto_fin_curso_kmeans_ga_jssp_final_revisado.docx.
- Documentación técnica utilizada por la implementación: scikit-learn, sklearn.cluster.KMeans.

Anexo A. Correspondencia con observaciones del profesor

Observación	Respuesta incorporada en el trabajo
-------------	-------------------------------------



Observación	Respuesta incorporada en el trabajo
Definir K y centroides	K = 3; KMeans con k-means++, n_init = 10 y semillas registradas.
Definir población y generaciones	20 individuos y 50 generaciones en todas las configuraciones.
Definir porcentaje aleatorio	A-D especifican proporciones entre 100%, 30% y 20% aleatorio.
Explicar lista restringida	Se incluye como mecanismo de variación controlada de órdenes base.
Comparar configuraciones y casos	A-D evaluadas en Base, Mediano y Grande con tres semillas.
Resultados y evidencia	CSV, Excel y figuras incorporados; capturas fechadas quedan para el paquete de entrega.
IA y licencia	Bitácora y texto de licencia académica incluidos en este informe.

